Cimple

Brugerdokumentation

Cimple er et begyndervenligt programmeringssprog med C-lignende syntaks. Det er designet til at gøre det nemmere at lære at programmere, uden at bekymre sig om svære koncepter som pointere og manuel hukommelsesstyring.

1. Datatyper

En datatype fortæller sproget, hvilken slags værdi en variabel indeholder.

Nøgleord	Beskrivelse	Eksempel
`integer`	Heltal (32-bit)	`integer x = 5;`
`double`	Decimaltal (64-bit)	`double pi = 3.14;`
`string`	Tekst	`string navn = "Anna";`
`boolean`	Sand/falsk-værdi	`boolean ok = true;`
`void`	Ingen returværdi (kun funktioner)	`void minFunktion() {...}`

2. Variabler

Variabler erklæres med en datatype, et navn og en startværdi. Startværdien kan også være et udtryk

```
integer alder = 17;
string besked = "Hej verden";
double pris = 49.95 + 3.2;
boolean aktiv = true;
boolean sammenligning = 5 < 8;
```

Du kan ændre en variabels værdi efter den er erklæret, men kun til samme datatype:

```
alder = 18;
```

3. Operatorer

3.1 Aritmetiske operatorer

Operator	Beskrivelse	Eksempel
`+`	Addition	`integer sum = 3 + 2;`
`-`	Subtraktion	`integer dif = 5 - 1;`
`*`	Multiplikation	`integer prod = 4 * 2;`

Operator	Beskrivelse	Eksempel
/	Division	double q = 10 / 4;
MOD	Rest ved division	integer r = 10 MOD 3;

3.2 Sammenlignings operatorer

Operator	Beskrivelse	Eksempel
==	Lig med	boolean eq = 5 == 7;
!=	Ikke lig med	boolean noteq = 8 != 5;
<	Mindre end	boolean less = 6 < 4;
>	Større end	boolean great = 8 > 2;
<=	Mindre end eller lig med	boolean leq = 5 <= 5;
>=	Større end eller lig med	boolean geq = 6 >= 1;

3.3 Logiske operatorer

Operator	Beskrivelse	Eksempel
AND	Og	boolean a = true AND false
OR	Eller	boolean b = true OR true
!	Ikke (negation)	boolean c = !true

4. Kontrol-strukturer

4.1 If / else

Udfører kode baseret på en betingelse:

```
if(x > 10) {
    print("Stort tal");
} else {
    print("Lille tal");
}
```

4.2 While-løkke

Gentager en kodeblok, så længe betingelsen er sand:

```
integer i = 0;
while(i < 5) {
    print(i);
    i = i + 1;
}
```

5. Arrays

Et array gemmer flere værdier af samme type.

Erklær med værdier:

```
integer tal[5] = [1, 2, 3, 4, 5];
```

Erklær tomt array med størrelse:

```
integer tal[10];
```

Tilgå eller sæt en værdi:

```
tal[0] = 99;
print(tal[0]);
```

> 💡 *Det første element har altid indeks 0.*

6. Funktioner

En funktion defineres med en returtype, et navn, parametre og et return-udtryk:

```
integer plusTo(integer a, integer b) {
    return a + b;
}
```

En funktion uden returværdi bruger void:

```
void sigHej(string navn) {
    print("Hej, " + navn);
    return 0;
}
```

Kald en funktion sådan:

```
integer resultat = plusTo(3, 4);
sigHej("Maria");
```

Der skal altid en main() funktion til for at køre programmet

```
void main() {
    print("Test program”);
    return;
}
```

7. Print

print bruges til at vise output i terminalen:

```
string s = "Hej verden";
print(s);
```

Her vil der blive printet ”Hej verden” i terminalen

8. Kommentarer

Kommentarer forklarer typisk kode og ignoreres af sproget:

```
/* Dette er en kommentar */

/*
   Dette er en
   kommentar over flere linjer
*/
```

9. Samlet eksempel

```
integer fakultet(integer n) {
    integer resultat = 1;
    while(n > 1) {
        resultat = resultat * n;
        n = n - 1;
    }
    return resultat;
}

void main(){
   integer svar = fakultet(5);
   print("5! = ", svar);
   return;
}
```

10. Operatorprioritet

Operatorer evalueres i denne rækkefølge (høj til lav):

Prioritet	Operator(er)
1 (højest)	( )
2	!
3	*, /, MOD
4	+, -
5	<, <=, >, >=
6	==, !=
7	AND, OR
8 (lavest)	= (tildeling)

11. Vigtige regler

- Alle udsagn afsluttes med ;
- Kodeblokke afgrænses med { }
- true og false skrives med små bogstaver
- Sproget er case-sensitiv — x og X er forskellige variabler
- Division med 0 er ikke tilladt og stopper programmet
- En variabel skal deklareres før den bruges
- Alle elementer i et array skal have samme datatype